



Industrial Parts Anomaly Detection

/// Jayden Chung & Pranav Thammana

Problem Statement



Industrial manufacturing relies heavily on consistent product quality, yet even small defects in components can lead to equipment failures, safety hazards, or costly product recalls

Traditional inspection methods struggle to scale with high production volumes and often fail to detect subtle, rare anomalies

Address the challenge of automated anomaly detection in industrial parts using convolutional autoencoders (CAEs)

Unlike supervised models that require labeled defect images, convolutional autoencoders can be trained exclusively on defect-free (“normal”) product images.

Methods

Encoder

Compresses the input into its essential features

Decoder

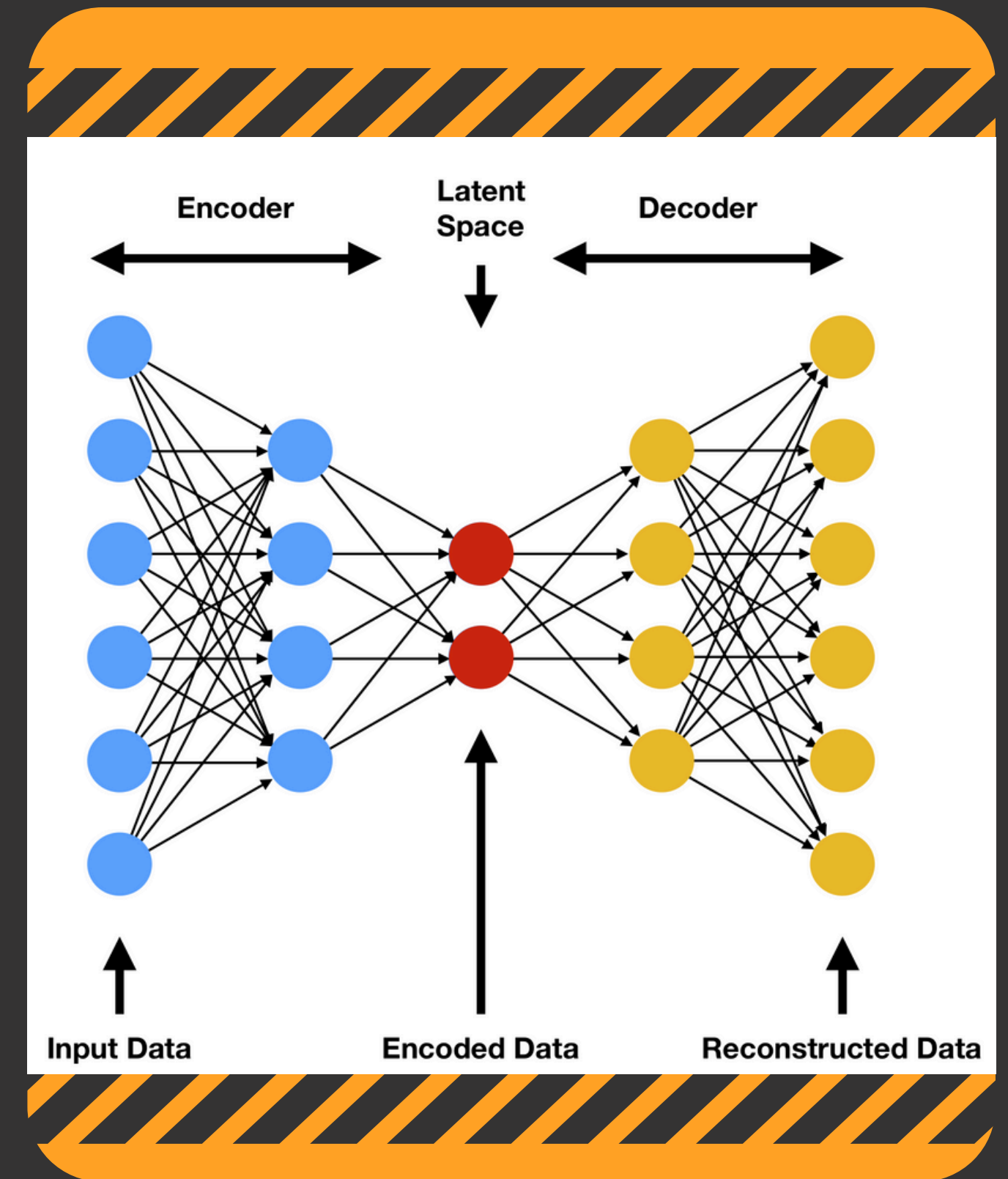
Rebuilds original input from the compressed representation

Autoencoder

Learns to reproduce its input by passing it through a bottleneck

VGG Heatmap

Highlights where the model is looking in the image



```
def build_unet_autoencoder(
    img_height=IMG_HEIGHT,
    img_width=IMG_WIDTH,
    channels=CHANNELS,
    latent_dim=32  # try 32, 16, etc.
):
    inputs = layers.Input(shape=(img_height, img_width, channels))

    # ----- ENCODER -----
    x = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs)
    x = layers.MaxPooling2D(2)(x)  # H/2
    x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
    x = layers.MaxPooling2D(2)(x)  # H/4
    x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
    x = layers.MaxPooling2D(2)(x)  # H/8

    h0 = img_height // 8
    w0 = img_width // 8

    x = layers.Flatten()(x)
    x = layers.Dense(latent_dim, activation='relu')(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(h0 * w0 * 64, activation='relu')(x)
    x = layers.Reshape((h0, w0, 64))(x)
```

Encoder

```
× × × ×
× × × ×
× × × ×
× × × ×
```

Learns what normal parts look like and encodes only those patterns

Since defects are not used in training, they do not map well into the latent space

```
# ----- DECODER -----
x = layers.Conv2DTranspose(64, 3, strides=2,
                           padding='same',
                           activation='relu')(x) # H/4
x = layers.Conv2DTranspose(32, 3, strides=2,
                           padding='same',
                           activation='relu')(x) # H/2
x = layers.Conv2DTranspose(32, 3, strides=2,
                           padding='same',
                           activation='relu')(x) # H

outputs = layers.Conv2D([channels, 3,
                        padding='same',
                        activation='sigmoid'])(x)

model = models.Model(inputs, outputs,
                     name="cable_autoencoder_weak")
return model
```

Decoder

```
x x x x
x x x x
x x x x
x x x x
```

Mirror image of the encoder

Decoder only knows how to reproduce “good” images

Higher reconstruction error in defect areas


```
autoencoder_bottle.compile(optimizer='adam',
                           loss=combined_ssim_mse_loss)

# Train the model
from sklearn.model_selection import train_test_split

# split good images into train (for AE) and validation (for threshold)
X_train_temp, X_val_threshold = train_test_split(
    X_train, test_size=0.1, random_state=42
)

X_train_ae, X_val_ae = train_test_split(
    X_train_temp, test_size=0.2, random_state=42
)

history = autoencoder_bottle.fit(
    X_train_ae, X_train_ae,
    epochs=20,
    batch_size=32,
    shuffle=True,
    validation_data=(X_val_ae, X_val_ae)
)

print("Autoencoder training complete.")
```

Autoencoder

x x x x
x x x x
x x x x
x x x x

Learn a compressed
representation of normality

Defects never appear in training
so model never learns them

During testing, defects produce
large reconstruction errors

```

# -----
# 1. Multi-scale VGG Feature Error Heatmap (no normalization here)
# -----
def get_feature_error_map(img, img_rec):
    """
    Returns a raw (unnormalized) heatmap at full image resolution.
    img, img_rec are (H, W, 3) arrays in [0,1].
    """

    # preprocess inputs for VGG
    x = preprocess_input((img * 255.0).astype('float32')[np.newaxis, ...])
    xr = preprocess_input((img_rec * 255.0).astype('float32')[np.newaxis, ...])

    # multi-scale VGG outputs (block3_conv3 ≈32x32, block4_conv3 ≈16x16)
    f3_true, f4_true = feature_extractor_multi.predict(x, verbose=0)
    f3_rec, f4_rec = feature_extractor_multi.predict(xr, verbose=0)

    # compute feature MSE at each scale
    err3 = np.mean((f3_true - f3_rec)**2, axis=-1)[0] # (H3, W3)
    err4 = np.mean((f4_true - f4_rec)**2, axis=-1)[0] # (H4, W4)

    # upsample both to input resolution
    err3_up = tf.image.resize(err3[..., None], (IMG_HEIGHT, IMG_WIDTH), method='bilinear')
    err4_up = tf.image.resize(err4[..., None], (IMG_HEIGHT, IMG_WIDTH), method='bilinear')

    # combine scales
    err_combined = 0.5 * err3_up + 0.5 * err4_up
    return err_combined # <-- NO normalization here!

```

VGG Heatmap

Shows which parts of the image
activated the convolutional
layers the most

Red / yellow = high importance
Blue / no color = low
importance

Overall test accuracy: 0.8795

Confusion matrix (rows = true, cols = pred):

```
[[19  1]
 [ 9 54]]
```

Classification report:

	precision	recall	f1-score	support
good	0.68	0.95	0.79	20
defective	0.98	0.86	0.92	63
accuracy			0.88	83
macro avg	0.83	0.90	0.85	83
weighted avg	0.91	0.88	0.89	83

Results

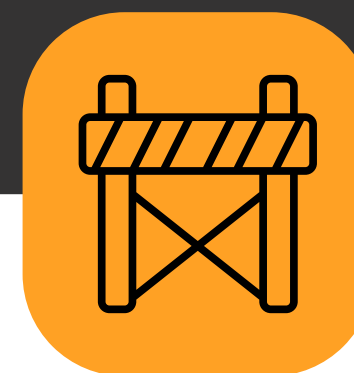


Challenges



Error Function

Evaluating test images
using the reconstructed
image



Other Parts

Difficult for autoencoder
to recreate high
definition images



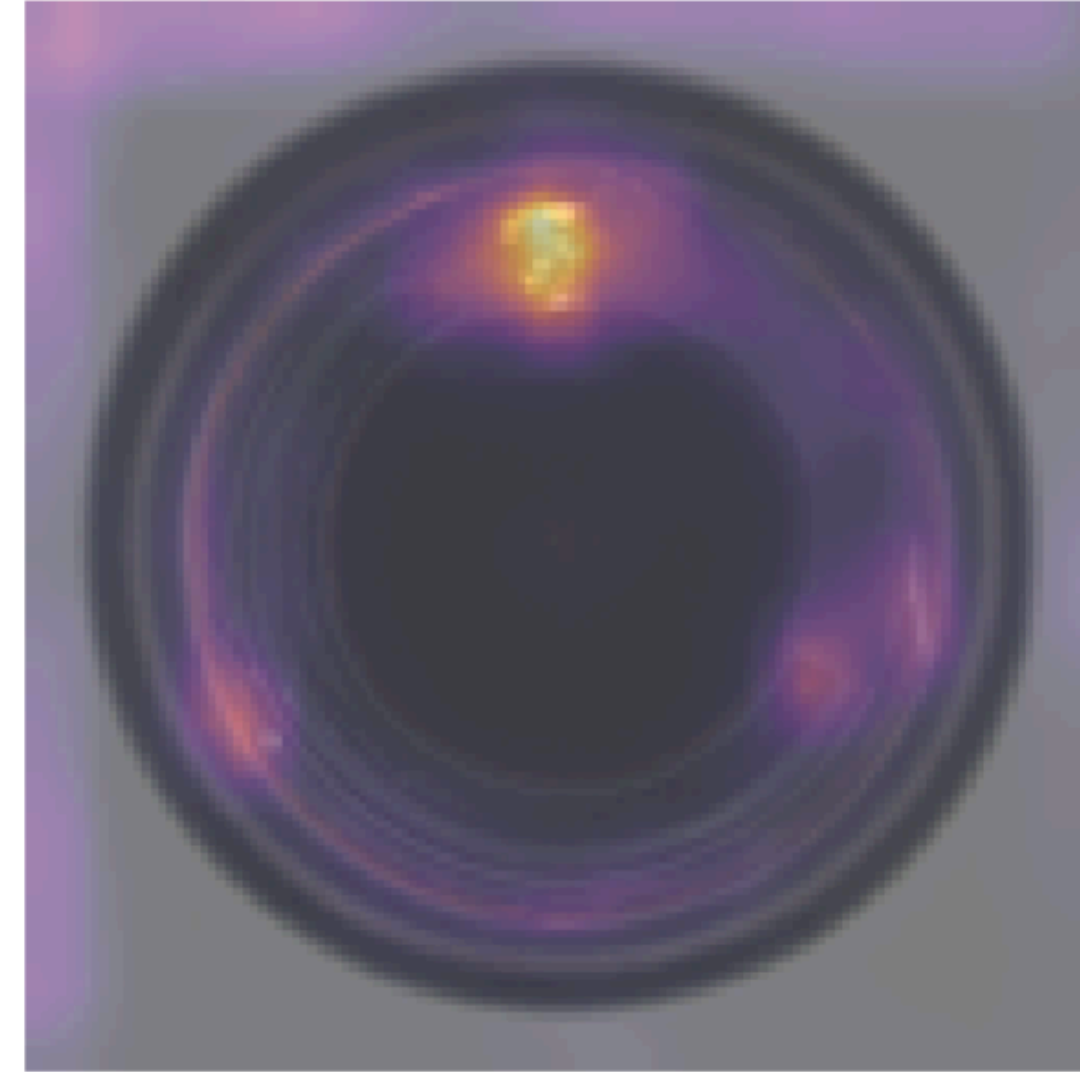
Original (ID 48)
Pred: Defective



Reconstructed
VGG error: 2605.1460
Threshold: 2183.5793



Multi-scale VGG Feature Error Heatmap



Conclusion